

**Московский государственный технический
университет им. Н.Э. Баумана**

**Факультет «Информатика и системы управления»
Кафедра ИУ5 «Системы обработки информации и управления»**

Курс «Парадигмы и конструкции языков программирования»

**Отчет по РК №2
Вариант запросов В
Вариант предметной области 6**

**Выполнил:
студент группы ИУ5-33Б
Розанов Федор**

**Проверил:
преподаватель каф. ИУ5
Гапанюк Ю. Е.**

Москва, 2025 г.

Листинг кода

main.py

```
class Department:
    def __init__(self, dept_id: int, name: str):
        self.dept_id = dept_id
        self.name = name

    def __repr__(self) -> str:
        return f"Department(id={self.dept_id}, name='{self.name}')"

class Employee:
    def __init__(self, emp_id: int, last_name: str, salary: int, dept_id: int):
        self.emp_id = emp_id
        self.last_name = last_name
        self.salary = salary
        self.dept_id = dept_id

    def __repr__(self) -> str:
        return (f"Employee(id={self.emp_id}, last_name='{self.last_name}', "
                f"salary={self.salary}, dept_id={self.dept_id})")

# класс-связка для многие-ко-многим
class EmpDept:
    def __init__(self, emp_id: int, dept_id: int):
        self.emp_id = emp_id
        self.dept_id = dept_id

    def __repr__(self) -> str:
        return f"EmpDept(emp_id={self.emp_id}, dept_id={self.dept_id})"

def query_employees_with_a(employees, departments):
    return [
        (e.last_name,
         next(d.name for d in departments if d.dept_id == e.dept_id))
        for e in employees
        if e.last_name.startswith("A")
    ]

def query_min_salary_by_department(employees, departments):
    dept_to_salaries = {d.dept_id: [] for d in departments}
    for e in employees:
        dept_to_salaries[e.dept_id].append(e.salary)

    result = [
        (next(d.name for d in departments if d.dept_id == dept_id), min(sals))
        for dept_id, sals in dept_to_salaries.items()
        if sals
    ]
    result.sort(key=lambda x: x[1])
    return result

def query_many_to_many_pairs(employees, departments, links):
    emp_by_id = {e.emp_id: e for e in employees}
    dept_by_id = {d.dept_id: d for d in departments}
```

```

pairs = [
    (emp_by_id[l.emp_id].last_name, dept_by_id[l.dept_id].name)
    for l in links
]
pairs.sort(key=lambda x: x[0])
return pairs

```

test.py

```

import unittest
from main import (Department, Employee, EmpDept,
                  query_employees_with_a,
                  query_min_salary_by_department,
                  query_many_to_many_pairs)

class TestQueries(unittest.TestCase):
    def setUp(self):
        self.departments = [
            Department(1, "Отдел кадров"),
            Department(2, "Бухгалтерия"),
            Department(3, "IT отдел"),
        ]
        self.employees = [
            Employee(1, "Александров", 80000, 1),
            Employee(2, "Иванов", 60000, 1),
            Employee(3, "Андреев", 90000, 2),
            Employee(4, "Петров", 70000, 2),
            Employee(5, "Архипов", 120000, 3),
        ]
        self.links = [
            EmpDept(1, 1),
            EmpDept(2, 1),
            EmpDept(3, 2),
            EmpDept(4, 2),
            EmpDept(5, 3),
            EmpDept(3, 3),
        ]

    def test_employees_with_a(self):
        result = query_employees_with_a(self.employees, self.departments)
        last_names = [ln for ln, _ in result]
        self.assertEqual(sorted(last_names),
                          sorted(["Александров", "Андреев", "Архипов"]))

    def test_min_salary_by_department(self):
        result = query_min_salary_by_department(self.employees,
                                                  self.departments)
        # результат отсортирован по минимальной зарплате
        self.assertEqual(result[0][1], 60000) # самый маленький минимум
        self.assertTrue(all(isinstance(s, int) for _, s in result))

    def test_many_to_many_pairs(self):
        result = query_many_to_many_pairs(self.employees,
                                            self.departments,
                                            self.links)
        # пары отсортированы по фамилии
        last_names = [ln for ln, _ in result]
        self.assertEqual(last_names, sorted(last_names))

if __name__ == "__main__":

```

```
unittest.main()
```

Скриншот работы приложения:

```
Ran 3 tests in 0.002s
```

```
OK
```

```
Launching unittests with arguments python -m unittest C:\U
```

```
Process finished with exit code 0
```